PRODUCT REQUIREMENTS DOCUMENT

Restaurant Management System

Cross-Platform Flutter + Firebase Platform

Version	1.2
Update	Firebase-managed application update delivery and release orchestration
Frontend	Flutter / Dart
Backend	Firebase Authentication, Cloud Firestore, Cloud Functions, FCM, Cloud Storage, Remote Config / Hosting where appropriate
Local persistence	SQLite through Drift or equivalent
Target platforms	Windows, macOS, Linux, Android, iOS, Web (subject to platform compatibility)
Architecture	Offline-first, role-based, multi-device, multi-branch

Purpose

Define product behavior, architecture, business rules, offline synchronization, safe release management, and acceptance criteria for a production-oriented restaurant management system. Version 1.2 adds a Firebase-backed control plane and artifact-delivery workflow for application updates, with platform-specific installation constraints.

1. Updated Product Vision

The system must operate as a resilient restaurant management platform that continues supporting essential restaurant operations during internet outages and can be upgraded safely without compromising business data or operational continuity.

The application must support connected and disconnected operating modes. Local data must be persisted reliably, pending operations must synchronize when connectivity returns, and conflicts must be resolved according to explicit business rules.

1.1 Updated Objectives

- Support offline order creation and local order management.
- Persist pending transactions locally before acknowledging successful local saves.
- Synchronize offline transactions automatically when connectivity returns.
- Prevent duplicate orders, payments, stock movements, and settlements during retries.
- Support offline kitchen operation and recoverable local printing.
- Allow delivery staff to record eligible delivery updates while disconnected.
- Distinguish locally recorded operations from server-confirmed operations.
- Provide conflict detection, resolution, retry, and synchronization monitoring.
- Support application updates, backend deployments, and local schema migrations.
- Prevent incompatible releases from corrupting local or cloud data.
- Support staged rollouts, migration recovery, and controlled rollback.
- Preserve financial auditability and access-control enforcement across offline and online workflows.

2. Offline-First System Architecture

2.1 Architectural Principle

The application must not depend on a continuously available internet connection for every user interaction. For supported operations, Flutter should read from and write to a local persistent database first. A synchronization engine then transfers pending changes to Firebase.

The local database is the operational data source while disconnected. Cloud Firestore and trusted backend operations become authoritative for server-confirmed records and financial state once synchronization succeeds. Not every operation is safe offline; the system must define which operations can be completed locally and which require server authorization.

2.2 Recommended Architecture

```
Flutter Cross-Platform Application
POS | Kitchen | Delivery | Manager | Accounting
|
Feature Logic / Domain Rules / Permissions
|
Repository Layer
|
Local Database + Transactional Outbox
|
Sync Engine / Conflict Resolver / Retry Policy
|
Connectivity Available
|
Firebase Authentication / Cloud Functions
|
Cloud Firestore

Local Print Adapter ---> Local or Network Printer
```

2.3 Local Persistence

Use a structured local database for operational data instead of relying exclusively on in-memory state or Firestore's client cache. Recommended: SQLite through Drift or an equivalent maintained Flutter library.

- Transactional local storage for business records and synchronization operations.
- Secure platform storage for credentials and encryption keys where applicable.
- Cached menu, pricing configuration, products, modifiers, taxes, and station assignments.
- Locally created orders, item snapshots, local payment records, kitchen tickets, and driver events.
- Pending outbox operations, local print jobs, conflict records, sync checkpoints, and schema versions.

2.4 Transactional Outbox

Every offline-capable operation must save both the local business change and its synchronization command in one local database transaction. For an offline order, atomically persist the order, item snapshots, locally calculated totals, unique operation ID, outbox entry, and required local print jobs. If any part fails, do not acknowledge the local save as successful.

Outbox fields: operationId, entityType, entityId, operationType, payload, schemaVersion, createdAt, lastAttemptAt, attemptCount, syncStatus, lastError, dependencyIds, and idempotencyKey.

Suggested states: pending, in_progress, synced, retryable_failure, conflict, requires_user_action, and permanently_rejected. Failed operations must remain recoverable; reaching a retry limit must not silently discard them.

2.5 Local and Server Identity

Generate globally unique client-side entity IDs, such as UUIDs. Human-readable order numbers may be provisional offline and assigned canonically by a trusted backend later. Keep the stable entity ID when replacing a provisional number. Receipts and tickets must clearly distinguish provisional references from server-confirmed references.

3. Offline Capability Matrix

Maintain this as a versioned capability policy rather than scattering offline checks throughout individual screens.

Feature	Offline capability	Required behavior
POS	Full for approved basic operations	Create and edit eligible orders using cached configuration
Menu	Cached read	Use last synchronized menu and pricing
Customers	Cached read / local creation	Synchronize new or updated records later
Cash payments	Local recording	Mark locally recorded and pending synchronization
External card payments	Integration-dependent	Require supported terminal workflow; never claim unverified success
Kitchen	Local queue	Process locally available orders and persist status changes
Printing	Conditional	Print through supported local integration
Delivery assignment	Restricted	Use valid cached or explicitly permitted provisional assignments
Driver updates	Queueable	Persist local events and synchronize later
Customer collection	Queueable with safeguards	Record locally; financial confirmation remains pending
Driver settlement	Restricted	Draft offline; final settlement requires trusted validation
Inventory	Conditional	Record local movements and flag concurrent conflicts
Reports	Cached / limited	Show data freshness and last sync time
User management	Online only	No offline permission changes
Refunds	Restricted	Require approved online workflow unless explicitly authorized otherwise

Feature	Offline capability	Required behavior
Configuration	Cached read	Changes require authorized server workflow

4. Offline POS Requirements

4.1 Offline Order Creation

- Detect disconnected state and continue using the last synchronized menu.
- Read cached product prices, modifiers, taxes, and applicable rules.
- Save the order, item snapshots, totals, local status, unique ID, and outbox command atomically.
- Display saved_locally_pending_sync, not a misleading server-confirmed status.
- Continue kitchen processing and local printing where supported.
- Synchronize on reconnection and update the order with the server-issued reference.

4.2 Offline Menu and Price Rules

Store a versioned snapshot containing products, modifiers, prices, taxes, availability, station assignments, permitted discount policies, branch identity, configuration version, and last successful synchronization time.

If the cache exceeds a configurable maximum age, warn the cashier and apply restaurant policy: continue selling, restrict selected products, or require manager authorization. Never silently alter an accepted offline order total after a pricing discrepancy is detected.

4.3 Offline Cash Payments

Cash transactions may be recorded offline when the cashier has a valid cached session and the terminal is authorized for offline use. Record order ID, local payment ID, amount, currency, method, user, device, local timestamp, and sync status. A locally recorded payment is not server-confirmed until validated after synchronization.

4.4 Offline Editing and Cancellation

Offline edits and cancellations must create auditable events rather than deleting history. If another device has already modified or completed the order, synchronization must detect the conflict. Financially significant conflicts require review rather than automatic last-write-wins.

4.5 Offline Order Numbering

Use stable unique order IDs, provisional human-readable references unique to the device or terminal, trusted server-issued numbering, and traceability between local and canonical references.

5. Offline Kitchen Requirements

5.1 Local Kitchen Queue

The kitchen must receive tickets from cloud orders and eligible locally created orders. Tickets must survive restarts and include stable ticket ID, order ID, provisional/confirmed reference, product and modifier snapshots, instructions, station, local timestamp, sync status, and preparation state.

5.2 Offline Kitchen State Changes

Persist status updates locally before acknowledging them. Record event ID, ticket ID, previous and new status, user, device, local timestamp, and sync status. Detect incompatible transitions made by multiple disconnected devices.

5.3 Kitchen Printing During Outages

Print locally created tickets when a supported local printer is available. Track ticket creation, print queueing, dispatch, confirmation where supported, and failure independently. Keep the KDS operational if printing fails. Do not automatically reprint every completed ticket after reconnecting.

6. Offline Delivery Requirements

6.1 Driver Application

Cache assigned deliveries and the minimum data required for authorized delivery tasks: order reference, customer details, address and instructions, order summary, expected collection amount, assignment state, and last synchronized assignment data. Restrict and protect cached personal information.

6.2 Offline Delivery Events

Allow eligible updates such as pickup, delivery attempt, delivered, failed, failure reason, and collection recorded. Save each event locally with a unique ID and queue it for synchronization. Clearly distinguish local recording from server confirmation.

6.3 Reassignment Conflicts

If a delivery is reassigned while the original driver's device is offline, validate the assignment on reconnection. Do not automatically accept an offline completion if the order was reassigned, cancelled, or otherwise made ineligible. Preserve conflicting events for investigation.

6.4 Offline Collections

An offline collection record must include amount, currency, payment method, order ID, driver ID, device ID, and unique collection transaction ID. It is a locally recorded claim, not proof that the restaurant received the money. Settlement approval and authoritative reconciliation require server-side validation.

7. Synchronization Engine

7.1 Responsibilities

- Detect connectivity changes and confirm actual backend reachability.
- Schedule synchronization and process pending operations in dependency order.
- Retry transient failures with exponential backoff and jitter.
- Preserve operation IDs across retries and enforce idempotency.
- Detect conflicts, record server acknowledgments, and update local states.
- Resume after restart and surface permanent failures.
- Expose synchronization health to authorized users.

7.2 Synchronization Order

Respect dependencies: create a customer if needed; submit the order; synchronize associated payment records; synchronize kitchen events; synchronize delivery assignment or status events; synchronize eligible collection records; refresh server-confirmed summaries. Operations on the same entity must preserve required ordering.

7.3 Retry Policy

Retry timeouts, transient network failures, and retryable service errors using configurable exponential backoff with jitter. Do not retry permanent errors indefinitely. Keep blocked operations visible with actionable reasons.

7.4 Idempotency

Every command must carry a stable idempotency key. The trusted backend must persist operation receipts and return the existing result for duplicate requests. Apply this to order creation, payments, refunds, tickets, assignments, collections, settlement allocations, inventory movements, and print jobs where appropriate.

7.5 Independent Status Dimensions

Use separate business status, payment status, settlement status, print status, and synchronization status. An order may be preparing while pending_sync. Do not overload the order lifecycle to represent connectivity.

7.6 Conflict Resolution

Entity	Required strategy
Menu configuration	Server-authoritative; refresh stale snapshots
Order creation	Idempotent creation using stable ID
Order items	Revision checks and explicit conflict handling
Kitchen status	Validate legal transitions and preserve event history
Driver assignment	Server-authoritative active assignment
Delivery events	Validate current assignment and order state
Cash payments	Append-only transaction records with deduplication
Driver collections	Append-only records with deduplication and reconciliation
Settlements	Trusted validation; never last-write-wins
Inventory	Append-only movements; explicitly resolve stock conflicts
Roles and permissions	Server-authoritative; no offline mutation
Reports	Refresh from authoritative aggregates

7.7 Synchronization Monitoring

Show connectivity, last successful sync, pending and failed operation counts, unresolved conflicts, and current sync activity. Provide authorized managers with a diagnostics screen and actionable error messages without exposing sensitive payloads.

8. Offline Security and Data Integrity

8.1 Local Access Control

- Protect local session data and sensitive caches.
- Lock the app after configurable inactivity.
- Clear or isolate user-specific caches on logout without silently destroying pending transactions.
- Prevent cross-user access to pending operations.
- Handle account deactivation and revoked permissions on reconnection.
- Never store Firebase service credentials in client data.

8.2 Local Data Protection

Use platform-appropriate encryption for sensitive local data where feasible. Do not hardcode encryption keys or store them alongside encrypted records. Document cached data, protection level, and effects of uninstall/reset.

8.3 Recovery

Define recovery for crashes, device shutdown during sync, database corruption, failed migrations, expired sessions, device replacement, and logout with pending transactions. Warn before actions that could destroy local business data. Approved backups must not duplicate transactions when restored.

9. Upgrade Management and Release Lifecycle

9.1 Upgrade Scope

Manage four coordinated but independent upgrade areas: client application binaries; local database schema; backend services (Cloud Functions, Security Rules, indexes); and cloud document/data schemas.

9.2 Version Management

Track appVersion, buildNumber, localSchemaVersion, apiContractVersion, minimumSupportedAppVersion, backendReleaseVersion, cloudDataSchemaVersion, and configurationVersion.

A trusted release manifest should include latest stable version, minimum supported version, release channel, update notes, mandatory/optional status, supported backend contract range, rollout cohort or percentage, activation time, and maintenance notices.

9.3 Upgrade Types

- Optional update: current version remains supported; installation may be deferred.
- Mandatory update: current version cannot perform specified protected operations; pending local data remains recoverable.
- Emergency update: addresses critical security or data-integrity issues with a documented recovery path.
- Configuration update: updates menus, printers, role layouts, or other server-managed configuration.
- Backend-only update: changes trusted functions, rules, indexes, or services while preserving client compatibility.

9.4 Platform-Specific Installation

Platform	Upgrade strategy
Windows	Signed installer or managed desktop update mechanism
macOS	Signed, appropriately distributed application update
Linux	Package manager, signed package, or managed deployment
Android	Supported app-store or managed distribution
iOS	App Store or supported enterprise distribution
Web	Versioned deployment, cache invalidation, and service-worker update management

Flutter does not provide a universal automatic binary-update mechanism for all platforms. Desktop auto-update infrastructure, mobile store deployment, and web release handling must be designed and tested separately.

10. Application Upgrade Workflow

10.1 Update Detection

When connected, check trusted release metadata at a controlled frequency. Determine update availability, mandatory status, schema migration needs, backend compatibility, and whether pending local operations make immediate restart unsafe.

10.2 Standard Upgrade Sequence

- Detect a new supported release and read the release manifest.

- Verify compatibility and update policy.
- Warn the user if restart is required.
- Inspect pending outbox entries and local transactions.
- Preserve unsynchronized operations and prepare recovery data.
- Obtain the platform-supported update package and verify authenticity.
- Install or activate the new version.
- Run local database migrations transactionally.
- Validate migrated data and synchronization metadata.
- Initialize the application and verify backend compatibility.
- Resume synchronization and record successful migration/startup.
- Report failures to monitoring.

10.3 Updates During Active Operations

Do not interrupt a cashier during a local database commit. Before restart, complete or safely abort transactions, persist outbox entries, warn about unsynchronized work, preserve active cash sessions, and avoid restarting during critical operations where supported. Mandatory updates may restrict new operations but must preserve access to pending-data recovery.

10.4 Failed Application Updates

Use a documented recovery strategy. Preserve the previous compatible version where the platform allows, retain pending transactions, record diagnostics, prevent incompatible clients from protected operations, and provide administrator recovery. Do not assume automatic binary rollback exists on every platform.

11. Local Database Migration Workflow

11.1 Schema Versioning

Every local database must include a schema version. Each release must declare supported starting versions and ordered migration steps. Migrations must be deterministic and tested against realistic datasets.

11.2 Migration Requirements

- Run before accessing the new schema.
- Use transactional changes where supported.
- Preserve business records, outbox entries, operation IDs, checkpoints, and audit history.
- Validate required fields and indexes.
- Update schema version only after successful completion.
- Support safe failure handling and guarded repeat execution.
- Do not process outbox commands while migration is incomplete.

11.3 Pre-Migration Validation

Check storage capacity, database integrity, pending operations, current schema, compatibility, and whether a recovery copy is required. If validation fails, stop migration and enter a safe recovery state.

11.4 Migration Failure Recovery

- Stop normal startup.
- Preserve the original database or recoverable snapshot.
- Record diagnostics and do not mark the migration complete.
- Offer supported retry or recovery procedures.

- Preserve pending offline operations.
- Notify authorized administrators when necessary.

12. Firestore and Backend Upgrade Workflow

12.1 Expand → Migrate → Contract

Expand: add optional fields/collections, deploy backward-compatible functions, create indexes, and preserve old-client behavior. Migrate: backfill documents and aggregates with checkpoints, monitor progress, and verify consistency. Contract: remove legacy fields only after supported clients no longer depend on them.

12.2 Cloud Data Migrations

Firestore requires explicit, versioned migration jobs for document transformations. Each job should include migration ID, source/target versions, checkpoints, batch controls, retries, idempotent transformations, progress, error records, verification, and completion status. Large migrations must be resumable.

12.3 Backend Compatibility

Test new backend releases against all supported client contract versions. Validate payload schema versions and reject unsafe or unsupported operations through trusted backend logic.

12.4 Rollback Strategy

Distinguish code rollback from data rollback. Revert compatible function deployments when appropriate, restore or adjust rules if needed, retain backward-compatible fields, and use compensating migrations for irreversible transformations. Configure and test appropriate Firestore backup and recovery options; a backup is not a substitute for a tested rollback plan.

13. Feature Flags and Staged Rollouts

13.1 Feature Flags

Use server-managed flags for new POS layouts, delivery assignment algorithms, settlement workflows, ticket formats, inventory engines, and reporting implementations. Support global, branch, role, cohort, and emergency-disable controls. Feature flags never replace backend authorization.

13.2 Staged Release

Release through development, automated testing, internal testing, staging, pilot deployment, gradual rollout, and general availability. Monitor crashes, sync failures, migration failures, duplicate orders, payment inconsistencies, and settlement discrepancies. Pause rollout when safety thresholds are exceeded.

13.3 Offline Update Awareness

A disconnected device cannot reliably know that a new mandatory update exists. Cache the last valid release policy, track its age, apply explicit stale-policy rules to high-risk operations, refresh on reconnect, and never discard unsynchronized records. Document allowed disconnected operation duration and reconnection restrictions.

14. Updated Data Model

14.1 Local Entities

localOrders: localId, serverId, provisionalOrderNumber, canonicalOrderNumber, localRevision, serverRevision, syncStatus, createdOffline, lastSyncedAt, lastSyncError.

outboxOperations: operationId, entityType, entityId, operationType, payload, schemaVersion, idempotencyKey, dependencyIds, syncStatus, attemptCount, createdAt, lastAttemptAt, lastError.

syncConflicts: conflictId, entityType, entityId, operationId, localSnapshot, serverSnapshot, conflictType, detectedAt, resolutionStatus, resolutionAction, resolvedBy, resolvedAt. Protect sensitive snapshots.

localPrintJobs: printJobId, sourceEntityId, documentType, printerId, status, attemptCount, createdAt, lastAttemptAt, lastError, reprintOfJobId.

localAppMetadata: appVersion, buildNumber, localSchemaVersion, lastSuccessfulSyncAt, lastReleaseCheckAt, cachedReleasePolicyVersion, migrationStatus, lastSuccessfulMigration, deviceId.

14.2 Server-Side Additions

Add releaseConfigurations, migrationJobs, migrationCheckpoints, supportedClientVersions, featureFlags, syncOperationReceipts, syncConflictRecords, and deviceRegistrations where needed.

Retain idempotency receipts for an appropriate period; do not expire them while delayed retries could still duplicate financial or business transactions.

15. Revised Acceptance Criteria

- AC-15 — Offline order creation: an eligible order is saved locally, shown as pending sync, and survives restart.
- AC-16 — Duplicate prevention: repeated synchronization creates only one server order.
- AC-17 — Offline payment integrity: cash payment is locally recorded and not falsely shown as server-confirmed.
- AC-18 — Offline kitchen: locally created tickets appear in the local kitchen workflow and survive restart.
- AC-19 — Offline printing: supported local printing works; failures do not duplicate orders or falsely mark jobs printed.
- AC-20 — Conflict handling: incompatible edits are detected; financial conflicts are not silently resolved last-write-wins.
- AC-21 — Offline driver updates: queued events are validated against the current assignment on reconnection.
- AC-22 — Offline settlement: locally recorded collections cannot finalize settlement without server validation.
- AC-23 — Migration preservation: outbox operations and idempotency keys survive database migration.
- AC-24 — Migration failure: the app enters safe recovery and does not delete the database or mark migration complete.
- AC-25 — Backend compatibility: supported old and new clients work during backend rollout.
- AC-26 — Mandatory update: unsupported clients follow update policy while pending local data remains recoverable.
- AC-27 — Rollback: critical releases can be paused and mitigated without blindly reverting financial records.
- AC-28 — Version visibility: authorized admins can inspect app/schema versions, last sync, pending operations, and migration state.
- AC-29 — Web update recovery: stale assets and service-worker updates do not lose offline transactions.
- AC-30 — Permission revocation: offline limitations apply and authorization is revalidated on reconnection.

16. Revised Definition of Done

Offline-capable features

- Persist eligible operations locally before acknowledging local success.
- Create transactional outbox entries and define dependencies.
- Implement idempotent retries and conflict rules.
- Expose synchronization status and handle restarts/interrupted operations.
- Protect cached sensitive data and test offline/reconnect/duplicate cases.
- Document operations requiring an online server.

Upgrade-related features

- Track application and schema versions.

- Validate compatibility before migration.
- Preserve pending business transactions.
- Provide tested failure recovery and non-destructive defaults.
- Support staging and controlled rollout.
- Document rollback limitations and test old/new client versions.
- Produce diagnostics without exposing secrets.

17. Additional Release Roadmap

Phase 0 — Technical Foundation

- Flutter architecture and separate Firebase environments.
- Authentication and permissions.
- SQLite persistence and local migration framework.
- Repository abstractions and transactional outbox.
- Synchronization engine and idempotency infrastructure.
- Version metadata, release configuration, logging, and diagnostics.

Exit criteria: a test entity can be created offline, survives restart, synchronizes after reconnection, and is not duplicated by retries.

Phase 1 — Core POS and Kitchen

- Menu caching, offline order creation, pricing snapshots.
- Local kitchen queue, online/offline workflows, local printing.
- Receipt/ticket reprinting and sync monitoring.

Exit criteria: an eligible order can be created during an outage, processed by the local kitchen workflow, and synchronized correctly.

Phase 2 — Delivery and Settlement

- Cached assignments, offline delivery events, offline collection recording.
- Conflict-aware reassignment, outstanding settlement queue, online settlement finalization.

Exit criteria: offline delivery activity remains traceable and cannot silently duplicate collections or settlements.

Phase 3 — Accounting, Administration, and Controlled Upgrades

- Cash sessions, payment allocation, refunds, approval workflows, audit logs.
- Release manifests, app update workflow, local migrations, backend compatibility.
- Feature flags, staged rollout, and rollback procedures.

Exit criteria: a new release can be deployed without losing pending offline operations or corrupting supported client data.

Phase 4 — Inventory and Advanced Operations

- Offline inventory movements and conflict-aware stock synchronization.
- Purchasing, supplier management, advanced reporting, multi-branch consolidation, automated delivery assignment.

Exit criteria: inventory and reporting remain consistent after concurrent operations and prolonged connectivity interruptions.

18. Final Architectural Requirements

- Local-first persistence for eligible operations.

- Transactional outbox for atomic local changes and sync commands.
- Idempotent synchronization that prevents duplicate business and financial records.
- Explicit distinction between local records and server-confirmed state.
- Independent business, payment, settlement, printing, and synchronization statuses.
- Conflict-aware handling of financial operations.
- Safe local schema migration preserving pending transactions.
- Backward-compatible backend releases for supported clients.
- Platform-specific, supported update mechanisms.
- Auditable collections, refunds, remittances, and adjustments.
- Controlled offline permissions and explicit stale-policy behavior.
- Tested recovery for crashes, interrupted sync, failed migrations, and deployment rollback.

Final product objective: deliver a cross-platform restaurant management system that continues essential operations during internet outages, synchronizes safely across multiple devices, preserves financial integrity, and supports controlled software evolution without losing business data.

Appendix A. Baseline Product Scope from PRD v1.0

The original product scope remains in force: POS/cashier workspace; kitchen display and ticket routing; customer receipts; dine-in, takeaway, and delivery orders; driver app and assignment; outstanding delivery settlement queue; accounting and cash sessions; menu and inventory; customer management; reports; notifications; configurable roles and layouts; multi-branch support; and Firebase-backed real-time synchronization.

Core business rule

Keep operational status, payment status, and financial settlement status separate while linking them through auditable transactions. A delivered order is not automatically financially settled, and a print job is not proof that an order was successfully submitted.

Version 1.2 — Firebase-Delivered System Updates

This section extends the existing offline-first and upgrade-management requirements. Firebase is the update control plane: it publishes release metadata, distributes or references approved artifacts, notifies clients, and records rollout state. It does not bypass operating-system, app-store, code-signing, or browser deployment rules.

1. Objective and scope

- Provide a centrally managed release channel so authorized administrators can publish an update once and make the applicable release available to restaurant terminals, manager devices, kitchen screens, and delivery-driver apps.
- Check update metadata at startup, after authentication, on a configurable interval while online, and when the app receives a release notification.
- Support optional, recommended, and mandatory updates, minimum supported versions, staged rollout, release notes, compatibility gates, and an auditable update history.
- Preserve pending offline orders, payments, delivery events, settlement records, print jobs, and the local synchronization outbox throughout download, restart, installation, and migration.
- Separate application binaries from configuration and feature flags. Remote Config may change safe runtime parameters; it must not be treated as a mechanism for replacing arbitrary Flutter executable code.

2. Firebase services and responsibilities

Firebase component	Required responsibility	Important constraint
Cloud Firestore	Store the signed/published release manifest, rollout eligibility, minimum version, release notes, compatibility ranges, and device update status.	Clients may read published metadata; only a restricted release-publisher identity may write production release records.
Cloud Storage	Store approved installer/package artifacts and platform-specific release assets, or host artifacts when the selected platform distribution flow permits it.	Use private buckets/rules or short-lived authenticated download access; never make sensitive artifacts publicly writable.
Cloud Functions	Validate release publication, calculate/assign rollout cohorts, issue authorized download metadata, record acknowledgements, and run cleanup/retention tasks.	Privileged actions must be server-side and authenticated; clients must not be able to promote their own release.
Firebase Cloud Messaging (FCM)	Send a lightweight “release available” notification or prompt clients to re-check the manifest.	Push is best-effort, not the source of truth; clients must also poll/check on launch and reconnect.
Firebase Remote Config	Control update-check intervals, rollout percentages, prompt behavior, and kill switches where suitable.	Do not store secrets or use Remote Config as a binary repository or executable-code updater.
Firebase Hosting	Serve the web build and versioned web assets for the browser client.	Web updates are deployed as a Hosting release; browsers may require reload/cache invalidation and cannot use native installer flows.

3. Release manifest contract

Maintain one immutable release record per platform, release channel, and version. A suggested manifest includes:

- releaseId; semanticVersion; buildNumber; platform; architecture; releaseChannel (development, beta, production); createdAt; publishedAt.
- releaseNotes; releaseStatus (draft, staged, active, paused, revoked); rolloutPercentage; targetBranchIds / deviceGroups where applicable.
- minimumSupportedVersion; minimumCompatibleBackendVersion; maximumCompatibleBackendVersion; localSchemaMin / localSchemaMax; requiredMigrationVersion.
- artifactStoragePath or approved distribution URL; artifactSizeBytes; SHA-256 checksum; signing/certificate metadata; release expiry.
- updatePolicy (optional, recommended, mandatory); forceAfter timestamp if policy requires it; rollbackTargetVersion where supported.
- publisher identity, approval/audit metadata, and release-manifest schema version.

The manifest must be validated and parsed defensively. Reject unknown mandatory fields, malformed versions, mismatched platform/architecture, expired releases, incompatible schema/backend ranges, and artifacts whose checksum or signature cannot be verified. The manifest itself should be protected from unauthorized edits; for stronger assurance, sign the manifest and verify its signature with a public key embedded in the client.

4. Update lifecycle

- **1. Publish:** CI builds the app for a target platform, runs tests and security checks, signs the artifact, uploads it to the approved destination, computes SHA-256, and creates a draft manifest.
- **2. Approve and stage:** An authorized release manager reviews release notes, migration requirements, compatibility ranges, and rollout scope; the release is published to a small cohort first.
- **3. Discover:** The client fetches the current manifest through authenticated Firebase access or a controlled public-read manifest. FCM may accelerate discovery but cannot be required for correctness.
- **4. Decide:** The client compares platform, installed version, channel, rollout cohort, minimum supported version, backend compatibility, and local schema requirements. It explains whether the update is optional, recommended, or mandatory.
- **5. Protect local work:** Before installation, the client confirms local persistence, records outbox counts, finishes or safely pauses local database transactions, and warns if the device has unsynchronized business operations. It must never discard pending work to make room for an update.
- **6. Download and verify:** Download to a temporary file, enforce expected size, validate SHA-256 and publisher signature where available, then atomically promote the verified file to the installer staging area.
- **7. Install through the platform:** Launch the approved platform-specific update path. If user or OS confirmation is required, clearly prompt the user; do not claim installation succeeded until the app verifies the installed build after restart.
- **8. Migrate and health-check:** Run transactional local database migrations, check app/backend compatibility, restore sessions safely, then resume synchronization. Record migration and startup health results.
- **9. Acknowledge:** When online, send installed version, device/app instance identifier, outcome, migration result, and failure category to the backend. Do not send unnecessary personal data.
- **10. Roll out or halt:** Increase rollout only after health thresholds are met. Pause or revoke the release on elevated crashes, failed migrations, sync errors, or support incidents.

5. Platform-specific delivery policy

Platform	Required delivery method	Product behavior
Android	Prefer Google Play managed updates for public production releases. For privately managed devices, use an authorized enterprise/MDM or approved installer flow; Firebase Storage may host the package only where installation policy permits.	App must clearly show source, version, permissions, and any OS installation confirmation. Do not promise silent install on ordinary consumer devices.
iOS / iPadOS	Use App Store distribution, TestFlight for testing, or an approved enterprise/MDM distribution model.	Firebase can announce the release and link to the approved distribution channel; it cannot silently replace the installed iOS app binary.
Windows / macOS / Linux	Download a signed platform-specific installer/update package from controlled storage or an approved release endpoint; use a maintained updater/installer with platform-specific signing and elevation handling.	Support resumable download, checksum/signature validation, graceful app shutdown, recovery instructions, and user consent when required.
Web	Deploy the Flutter web build through Firebase Hosting with versioned assets and appropriate cache headers.	Show an update/reload prompt when a new build is available. Never attempt a desktop/mobile binary installation from the web client.
Firebase App Distribution	Use for internal QA, tester groups, and pre-release validation.	Do not treat App Distribution alone as the production auto-update mechanism for all customers.

Implementation note: Firebase provides the release metadata, notification, storage, hosting, and orchestration building blocks. The actual binary installation mechanism must be selected per platform and distribution channel. Firebase Remote Config or Firestore alone cannot safely update arbitrary native Flutter code.

6. Offline behavior and reliability

- When offline, the app continues its supported restaurant workflows and displays the last successful update-check time. It must not block POS, kitchen, or driver operations merely because the update service is unreachable.
- Never enforce a mandatory update while the device is offline unless a previously cached, signed policy explicitly permits a safe local decision. Even then, preserve access to pending transaction review/export and a documented recovery path.
- Use exponential backoff with jitter for manifest checks and artifact downloads; support cancellation, retry, and resume where the platform download stack supports it.
- Keep current and staged versions separate. Do not overwrite a working installation with an incomplete download.
- Do not automatically downgrade a client to an older binary solely because a manifest changed. Rollback must be an explicit approved release action and must consider database forward/backward compatibility.
- If installation fails, retain the current working version where the platform permits, preserve local data, and show actionable diagnostics without exposing tokens or signed download URLs.

7. Security, authorization, and audit

- Restrict production publishing and promotion to a dedicated release-manager role, protected CI identity, or approved server-side workflow; require multi-person approval if the business risk warrants it.
- Apply least-privilege Firebase Security Rules and IAM. Clients must not write production manifests, change rollout percentages, or overwrite published artifacts.
- Require TLS, authenticated access where appropriate, artifact checksum verification, platform code-signature verification, and manifest signature verification for high-assurance deployments.
- Use Firebase App Check where supported to reduce abuse of Firebase-backed endpoints; App Check is defense-in-depth and does not replace authentication or authorization.
- Never embed service-account keys, private signing keys, long-lived privileged tokens, or other secrets in the Flutter client or manifest.
- Log release publication, pause/revoke actions, client discovery, download result, install result, migration result, and rollback decision with timestamps and non-sensitive device identifiers.

8. Data model additions

Collection / record	Core fields
releaseManifests/{releaseId}	platform, architecture, channel, version, buildNumber, status, rolloutPercentage, minSupportedVersion, backend/schema compatibility, artifact path, size, SHA-256, signature metadata, notes, publishedAt, publisherId.
deviceUpdateStatus/{deviceId}	appVersion, buildNumber, platform, branchId, channel, lastManifestCheckAt, availableReleaseId, downloadStatus, installStatus, migrationStatus, lastErrorCode, lastSeenAt.
releaseAuditEvents/{eventId}	releaseId, actorId, action, timestamp, affectedChannel/cohort, reason, approval reference.
updatePolicy/{policyId}	check interval, rollout guardrails, mandatory-update rules, allowed channels, maintenance window, emergency pause flag.

Use server timestamps for cloud audit events. Device identifiers must be scoped, resettable where appropriate, and documented in the privacy policy. Do not store customer order details in update telemetry.

9. Update UX requirements

- Display installed version, latest eligible version, release notes, update severity, download progress, verification state, and installation instructions.
- Offer Update now and Remind me later for optional updates. Mandatory updates must explain the deadline and the reason, and follow platform policy.
- Display a warning when unsynchronized operations exist; allow the user to inspect pending work. A policy must define when an update can proceed without connectivity and when it should be deferred.
- Provide a manager-only screen to view device versions and update status by branch, filter failed/outdated devices, pause a rollout, and open remediation guidance.

• Do not expose stack traces, authentication tokens, private storage paths, or sensitive backend errors to normal users.

10. Acceptance criteria — Firebase update delivery

ID	Acceptance criterion
AC-31	An authorized publisher can publish a release manifest for a specific platform/channel without allowing ordinary clients to modify it.
AC-32	Online clients discover eligible releases through a manifest check; FCM notification loss does not prevent eventual discovery.
AC-33	Clients reject an artifact when size, SHA-256, signature, platform, or compatibility checks fail.
AC-34	A release can be staged to a configurable cohort and paused or revoked without rebuilding the client.
AC-35	An update never deletes or silently drops unsynchronized orders, payments, kitchen events, delivery collections, settlements, or outbox commands.
AC-36	The app only installs updates through the approved platform-specific distribution mechanism and respects OS/app-store confirmation requirements.
AC-37	Local database migration failure triggers the documented recovery path and does not silently mark the update successful.
AC-38	After restart, the client reports the installed build and health status when online, and resumes synchronization safely.
AC-39	Web clients receive Firebase Hosting releases and can detect a newer build and prompt for a safe reload.
AC-40	Manager/release views show per-device update status and categorized failures without exposing secrets or customer business data.
AC-41	A simulated outage of Firestore, Storage, FCM, or the internet does not break already-supported offline restaurant workflows.
AC-42	Production release permissions, audit logs, artifact verification, rollout pause, and rollback/recovery are tested before launch.

11. Release and rollback runbook

• Build and test platform-specific artifacts in CI; run unit, integration, migration, offline-sync, and startup smoke tests.

• Sign artifacts, compute checksums, upload to a versioned immutable storage path, and publish a draft manifest.

• Approve the release; stage to internal QA and then a small production cohort.

• Monitor crash-free starts, sync success, migration failures, download/install failures, and update adoption.

• Pause rollout immediately if a guardrail is breached. Revoke the manifest for new downloads if necessary; already installed clients must remain safe.

• For a faulty app release, publish a compatible fixed version or an approved rollback build. Do not assume binary rollback also reverses database migrations.

• For backend changes, preserve the existing expand → migrate → contract approach and maintain compatibility with supported client versions during rollout.

• Document incident owner, customer communication, recovery steps, and the release decision in the audit trail.

12. Definition of Done additions

• Release manifest schema, Firebase Rules/IAM, Cloud Functions, artifact storage, and FCM behavior are tested in a non-production Firebase project.

• CI/CD can build, sign, checksum, upload, and publish a release without exposing signing secrets to clients.

• Each target platform has a documented and tested installer/distribution flow; unsupported silent-update behavior is not claimed.

• Offline data preservation, local schema migration, failure recovery, staged rollout, pause/revoke, and client health reporting pass acceptance tests.

- Operational dashboards, alert thresholds, release permissions, retention policy, and incident/rollback runbook are approved.

Version history

v1.1 — Added offline-first operations, synchronization, migration safety, and general upgrade management.
v1.2 — Added Firebase-managed release manifests, Cloud Storage artifact delivery, FCM update notifications, platform-specific installation constraints, staged rollout, verification, telemetry, and update acceptance criteria.